Robotics · Motors & Motion Control

Moving Precisely

A DC motor spins. A servo points. A stepper counts. Which one draws a perfect circle?

60 minutes **Pairs** **Circuit Lab or real kit**

DC motor	Servo	Stepper
spins, fast	points, 0-180°	counts steps
no idea where	holds an angle	2048 per turn
wheels	arms, grippers	printers, CNC

only the servo checks itself: DC and stepper never know where they are

Did you know?

The 28BYJ-48 stepper in your kit takes 2048 steps for one turn of the output shaft. That is a resolution of 0.176 degrees, finer than you can see.

Let's think

Draw a square, in the dark

The challenge

Close your eyes. Draw a 10 cm square on paper. Open them. It is not square, and the lines are not 10 cm.

Now do it again with your eyes open. Better, because you can **check as you go**.

That is the whole lesson

A DC motor is drawing with its eyes shut. A stepper counts its own moves. A servo has a tiny eye inside it that checks the angle hundreds of times a second.

Guess first

- Which motor is in a printer that must not drift?
- Which is in a toy car?
- Which is in a camera gimbal that must hold still?
- Which one would you put on a robot arm elbow?

No wrong answers yet. Write them down, and check at the end.

LEARN

Learning aims

Today I will learn to...

1.  Compare **DC, servo and stepper** motors and what each is for

2.  Explain how a **pulse width** sets a servo angle

3.  Drive a servo to an angle and a stepper **an exact number of steps**

4.  Pick the right motor for a job and **justify it**

Keywords

servo · stepper · PWM · pulse width · steps per revolution · open loop · torque · holding torque · gearbox

Objective 1 · Compare

Three motors, three jobs

What each one actually gives you

DC motor
spins, fast
no idea where

wheels

Servo
points, 0-180°
holds an angle

arms, grippers

Stepper
counts steps
2048 per turn

printers, CNC

only the servo checks itself: DC and stepper never know where they are

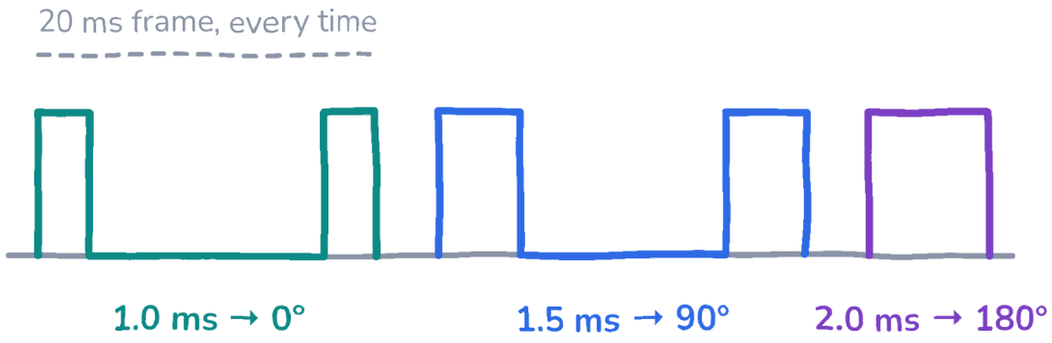
	DC	Servo	Stepper
Controls	speed	angle	position in steps
Range	endless	0 to 180°	endless, counted
Knows where it is	no	yes, inside	only if you count
Holds still under load	no	yes	yes, strongly
Needs a driver	yes	no	yes
Typical use	wheels	arms, steering	printers, CNC

Objective 2 · Servos

The pulse is the instruction

Width, not voltage, not count

the servo does not count turns, it reads pulse width



Inside the case

A small DC motor, a gearbox, a **potentiometer** on the output shaft, and a tiny controller that compares the pot reading with the pulse you sent.

It is a closed loop

Push the arm and the servo pushes back, because the pot tells it the angle is wrong. That is why a servo holds a load and a DC motor does not.

Buzzing?

The servo is hunting: it cannot reach the angle you asked for. Either something is jammed, or the 5 V rail is sagging.

Objective 3 · Steppers

Counting, not guessing

How a stepper moves

Four coils are energised in turn. The rotor jumps to line up with whichever coil is on. Each jump is one **step**, always the same size.

No feedback is needed, because the step size is set by the physical magnets, not by timing.

The 28BYJ-48 numbers

32 steps per turn of the *internal* rotor, then a **64:1 gearbox**.

32 × 64 = **2048 steps per output turn**. One step is 0.176°.

The catch: open loop

If the load is too heavy, the rotor **skips a step** and nothing tells you. The code still thinks it is at 90°. The arm is at 83°.

This is why 3D printers have a home switch: the only way to be sure is to go back to a known place.

Slower is stronger

Steppers lose torque as they speed up. If yours is skipping, add a longer delay between steps before you blame the motor.

In the real world

Open your eyes and spot them



DC motor

A pedestal fan, a drill, a toy car. You want it to go round; nobody cares exactly where it stops.



Servo

The flaps on a model aeroplane, a camera gimbal, the steering of an RC car. Hold this angle, against wind.

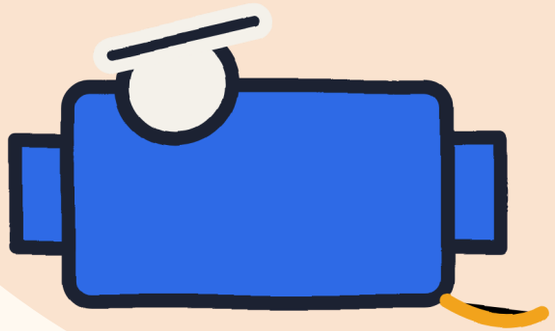


Stepper

A 3D printer, a CNC router, the paper feed in a photocopier. Move exactly this far, every time.

Part 2 · Make it

Now let's drive them



A servo that points, then a stepper that counts out exactly one revolution.

30 minutes | Pairs | Circuit Lab or real kit

Class game

Be the stepper

How to play

1. Four students stand in a square. Each is a coil.
2. A fifth stands in the middle: the rotor, arm pointing out.
3. Coils light up in order 1, 2, 3, 4. The rotor must point at whoever is lit.
4. Speed up the calling until the rotor cannot keep up.

That failure is a **skipped step**, and it is exactly what a real stepper does when it is pushed too fast.

Round two

Now call them backwards: 4, 3, 2, 1. The rotor reverses. Nothing else changed.

Half stepping

Light two neighbours at once and the rotor points between them. That doubles your resolution, for free.

Before you wire

What you need



Kit, per pair

- SG90 servo (orange signal, red 5V, brown GND)
- 28BYJ-48 stepper + ULN2003 driver board
- Arduino Uno + USB cable
- Half-size breadboard, jumper wires



Power warning

One SG90 on the Arduino's 5 V is fine. **Two or more is not.** They pull current in spikes and reset the board.

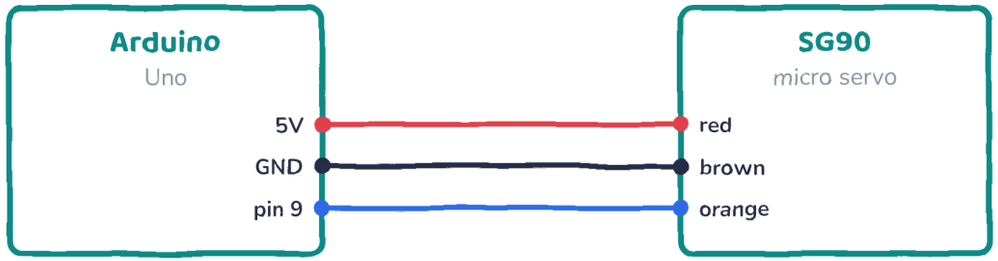
The stepper always takes its own 5 V supply through the ULN2003. Share the ground, never the 5 V rail.

Wiring · servo

Three wires, by colour

SG90 on a PWM pin

three wires, and the colours tell you which is which



pin 9 must be a PWM pin: 3, 5, 6, 9, 10 or 11

two or more servos need their own 5 V supply

Connections

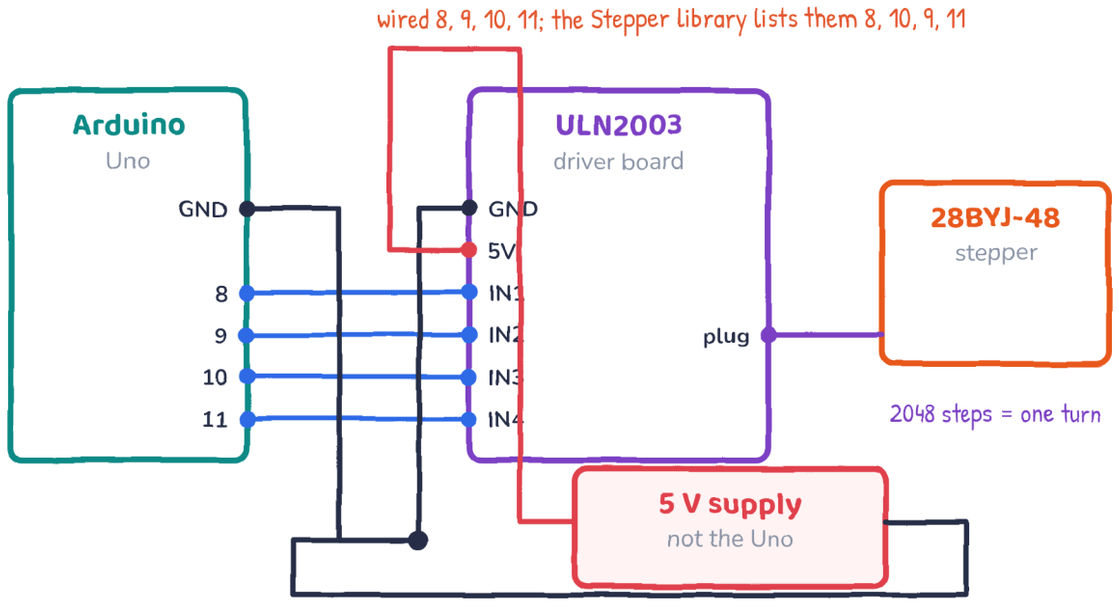
Arduino Uno	SG90 servo	Carries
5V	red	power for the motor inside
GND	brown	the shared reference
pin 9	orange	the pulse that sets the angle

Brown is always ground on a hobby servo. If the colours are black, red, yellow, black is ground.

Wiring · stepper

Four coils, four pins

28BYJ-48 through a ULN2003



Connections

Arduino Uno	ULN2003	Carries
8	IN1	coil 1
9	IN2	coil 2
10	IN3	coil 3
11	IN4	coil 4
(own 5 V)	5V	coil current, not from the Uno
GND	GND	the shared reference

Wire 8, 9, 10, 11 to IN1 to IN4. The sketch lists them 8, 10, 9, 11: that is not a typo.

MAKE

Build step 1 of 3

A servo that points

Do this

1. Signal to pin 9, red to 5V, brown to GND.
2. Upload the sweep. Watch it travel 0 to 180 and back.
3. Hold the arm gently. Feel it push back.
4. Now make it point at a number you type in the Serial Monitor.

Never force the arm

Turning a powered servo by hand strips the plastic gears. If you must reposition it, cut the power first.

```
#include <Servo.h>
Servo arm;

void setup() {
  arm.attach(9);
  Serial.begin(9600);
}

void loop() {
  for (int a = 0; a <= 180; a += 2) {
    arm.write(a);              // degrees, not pulses
    delay(15);                 // give it time to arrive
  }
  for (int a = 180; a >= 0; a -= 2) {
    arm.write(a);
    delay(15);
  }
}
```

Build step 2 of 3

A stepper that counts

Do this

1. ULN2003 IN1-IN4 to pins 8, 9, 10, 11.
2. Mark the shaft with tape so you can see one turn.
3. Run exactly 2048 steps. Does the mark come home?
4. Now run 1024. It should stop half way round.

Watch the order

The Stepper library wants pins in the order **8, 10, 9, 11** for this board. Wrong order means it vibrates and goes nowhere.

```
#include <Stepper.h>

const int STEPS = 2048;       // one full turn
Stepper motor(STEPS, 8, 10, 9, 11);

void setup() {
  motor.setSpeed(10);         // rev/min, keep it low
  Serial.begin(9600);
}

void loop() {
  motor.step(STEPS);          // one turn clockwise
  delay(1000);
  motor.step(-STEPS / 4);     // back a quarter turn
  delay(1000);
}
```

MAKE

Build step 3 of 3

A pointer that means something

Do this

1. Draw a dial on card: 0 to 100.
2. Tape a card arrow to the servo horn.
3. Map a sensor reading onto 0 to 180 degrees.
4. Now it is a gauge, not a toy.

map() is your friend

Every sensor has its own range. map() stretches it onto the range your output needs, in one line.

```
#include <Servo.h>
Servo needle;

void setup() {
  needle.attach(9);
}

void loop() {
  int raw = analogRead(A0);          // 0 to 1023
  int deg = map(raw, 0, 1023, 0, 180);
  needle.write(deg);
  delay(50);                         // smooth, not jumpy
}
```

Test like an engineer

Find the limits

Try this	What happens	What it teaches
Ask the servo for 200 degrees	It stops at its limit and buzzes	The range is physical, not a suggestion
Hold the servo arm still for 5 seconds	It gets warm	Holding torque costs current
Set stepper speed to 30 rev/min	It stalls or skips	Torque falls as speed rises
Load the stepper shaft with a finger	It loses steps silently	Open loop never knows it failed
Run two servos from Arduino 5 V	The board browns out	Motors need their own supply

Record the numbers

At what speed did your stepper first skip? That number is a property of your motor, and a real engineer writes it down.

Finished early?

Go further

1 

Work out **degrees per step** and prove it with a protractor

360 ÷ 2048. Measure 100 steps and check.

2 

Add **acceleration**: start slow, speed up, slow down

Fewer skipped steps and much less noise.

3 

Try **half stepping** and count the new steps per turn

4096. Twice the resolution, half the torque.

4 

Drive a servo with **raw pulses** and no library

digitalWrite, delayMicroseconds, repeat every 20 ms.

Check yourself

Can I really do it? Prove it.

1 Compare **DC, servo and stepper** motors

Proof: I picked the right one for three different machines

2 Explain how a **pulse width** sets a servo angle

Proof: I said what 1.5 ms means without looking

3 Drive a stepper **an exact number of steps**

Proof: my tape mark came back to the start

4 Pick the right motor for a job and **justify it**

Proof: I gave a reason, not just a name

Rate yourself: red = not yet · amber = nearly · green = yes, and I can show someone

Show what you know

What did we learn?

Five things to keep

- DC controls speed, servo controls angle, stepper counts.
- A servo reads pulse width, between 1 and 2 ms.
- A servo is closed loop; a stepper is not.
- 2048 steps is one turn of the 28BYJ-48.
- Steppers lose torque as they speed up.

Exit ticket

You are building a robot that must lift a cup to a shelf and hold it there. Which motor, and what happens if you choose wrong?

Next class

ROB-07: encoders. How a robot works out how far it has actually travelled, rather than how far it meant to.